

Основа обработки данных

Как только появились первые

механические и электромеханические машины для обработки переписей населения, возникла необходимость искать "глобальные параметры" и группировать данные (искать пары, тройки событий).

Современные алгоритмы Python для Задания 17 — это прямые наследники первых табуляторов Холлерита, обрабатывающих массивы чисел с перфокарт.



Блиц-опрос

Вопрос 1: Ищем глобальный минимум

Представьте, что в файле `17.txt` находится 10 000 целых чисел. Как с помощью встроенных функций Python быстро найти **самое маленькое число во всей этой последовательности**?

Блиц-опрос

Вопрос 1: Ищем глобальный минимум

Представьте, что в файле `17.txt` находится 10 000 целых чисел. Как с помощью встроенных функций Python быстро найти **самое маленькое число во всей этой последовательности**?

```
with open('17.txt') as f:
    a = [int(x) for x in f]

min_val = min(a) # Нашли минимальный элемент!
```

Зачем это нужно? В новых прототипах ЕГЭ (ФИПИ) часто требуется сравнивать свойства пары элементов именно с минимальным (или максимальным) элементом всего файла. Это называется **"Глобальный параметр"**.

Блиц-опрос

Вопрос 2: Тройки вместо пар

Раньше мы перебирали все смежные пары чисел с помощью перебора индексов: `for i in range(len(a) - 1)`.

Как должен измениться `range`, если теперь мы хотим перебрать все возможные **тройки** соседних элементов, и как к ним обратиться?

Блиц-опрос

Вопрос 2: Тройки вместо пар

Раньше мы перебирали все смежные пары чисел с помощью перебора индексов: `for i in range(len(a) - 1)`.

Как должен измениться `range`, если теперь мы хотим перебрать все возможные **тройки** соседних элементов, и как к ним обратиться?

```
# Вычитаем из длины массива размер "окна" минус один (3 - 1 = 2)
for i in range(len(a) - 2):
    x = a[i]
    y = a[i + 1]
    z = a[i + 2]
```

1: Чтение файла и генераторы

В задачах 17 самым эффективным методом загрузки данных является использование **генератора списков** (list comprehension). Это позволяет в одну строку считать весь файл, очистить строки от переносов и перевести их в целые числа `int`.

```
# Открываем файл '17.txt' и сразу создаем список  
with open('17.txt') as f:  
    a = [int(x) for x in f]  
  
# Теперь a - это готовый к работе массив  
print("Загружено элементов:", len(a))
```

2: Глобальные параметры (Минимум / Максимум)

Глобальным параметром называется значение, которое вычисляется для всей последовательности чисел и затем используется при проверке конкретных элементов. Это может быть минимальный элемент всего файла.

```
# Находим глобальный минимум ДО цикла проверки пар
```

```
min_val = min(a)
```

```
max_val = max(a)
```

Важно: Находить такие параметры внутри цикла перебора пар — **смертельная алгоритмическая ошибка**. Программа будет работать слишком долго (асимптотика $O(N^2)$).

3: Глобальные параметры с условием

Часто в задаче просят найти максимум *среди определенных чисел* (например, оканчивающихся на 3). Для этого можно использовать генератор прямо внутри функции `max()`:

```
# Глобальный максимум среди элементов, оканчивающихся на 3
```

```
max_3 = max(x for x in a if abs(x) % 10 == 3)
```

```
# Глобальный минимум среди трехзначных чисел
```

```
min_999 = min(x for x in a if 100 <= abs(x) <= 999)
```

Тут `x for x in a if ...` собирает на лету элементы массива, отвечающие условию, и передаёт их в функцию поиска минимума или максимума.

4: Особенности отрицательных чисел и `abs()`

Так как в 17 задании элементы могут быть в диапазоне `[-100 000; 100 000]`, проверка "оканчивается на цифру" через остаток от деления часто приводит к ошибкам.

Дело в том, что в Python оператор `%` для отрицательных чисел работает не как в C++: `-12 % 10` вернёт 8, а не 2.

```
# ОПАСНО (неправильно для отрицательных):
```

```
if x % 10 == 3: ...
```

```
# ПРАВИЛЬНО:
```

```
if abs(x) % 10 == 3: ...
```

5: Длина числа (Двухзначные, Трёхзначные)

Как проверить, что число трёхзначное?

Способ 1 (Математический): Проверить модуль числа.

```
if 100 <= abs(x) <= 999:  
    # Число трёхзначное
```

Способ 2 (Строковый): Преобразовать число по модулю в строку и измерить её длину. Это бывает удобнее для пяти-шестизначных чисел!

```
if len(str(abs(x))) == 4:  
    # Число четырёхзначное
```

6: Системы счисления (Встроенные функции)

В Python есть очень мощные функции перевода чисел в 2-ичную (`bin`), 8-ричную (`oct`) и 16-ричную (`hex`) системы счисления.

Все они возвращают **строку** с техническим префиксом (`0b`, `0o`, `0x`). Чтобы от него избавиться, берём срез `[2:]`.

Двоичная:

```
s2 = bin(x)[2:] # "10110"
```

Проверка окончания в восьмеричной системе:

```
if oct(x)[2:].endswith('3'):
```

Оканчивается на 3 в 8-ричной

7: Системы счисления (Любые основания)

А что если система счисления 3-ичная, 5-ичная или 7-ичная? Встроенных функций нет, нужно написать **свою короткую функцию** перевода числа n в систему с основанием b .

```
def to_base(n, b):  
    if n == 0: return '0'  
    res = ''  
    while n > 0:  
        res = str(n % b) + res  
        n //= b  
    return res  
  
if to_base(x, 5).endswith('2'):  
    # x в пятеричной оканчивается на 2
```

8: Сумма цифр числа

Чтобы найти сумму всех цифр стоящих в числе (например, для проверки "сумма цифр кратна 4"), проще всего конвертировать число в строку, перебрать символы (цифры), перевести их обратно в `int` и просуммировать:

```
# Магия генераторов:  
sum_digits = sum(int(d) for d in str(abs(x)))  
  
if sum_digits % 5 == 0:  
    # Сумма цифр числа x кратна 5
```

Важно! Функция `abs()` здесь снимает минус, иначе при переводе в строку первым символом будет '-', который не переводится в целое число (вызовет ошибку `'ValueError'`).

9: Функции `any()` и `all()`

Когда в условии задачи спрашивается "хотя бы одно число четко делится на 3" или "каждое число трехзначное", очень помогают функции `any(...)` и `all(...)`.

```
# Хотя бы одно число пары кратно 5  
cond1 = any(n % 5 == 0 for n in (x, y))
```

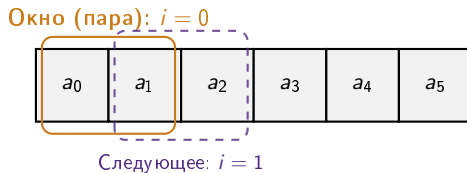
```
# Все числа тройки положительные  
cond2 = all(n > 0 for n in (x, y, z))
```

```
# Ровно два из трех чисел оканчиваются на 7  
cond3 = sum(abs(n) % 10 == 7 for n in (x, y, z)) == 2
```

В `cond3` мы используем математику `True = 1`, `False = 0`, поэтому сумму условий можно приравнять к двойке.

10: Пары элементов (Окно = 2)

В Задании 17 самое частое условие — работа с идущими подряд **парами** элементов (окно размера 2).

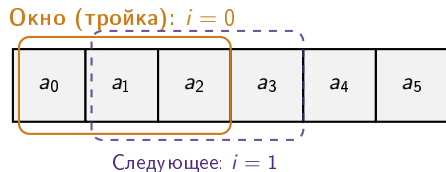


Длина цикла: чтобы не выйти за границы списка, нужно вычитать размер "окна" минус 1 из `len(a)`.

```
# Для пар (окно из 2 элементов: 2 - 1 = 1)
for i in range(len(a) - 1):
    x, y = a[i], a[i+1]
```

11: Тройки элементов (Окно = 3)

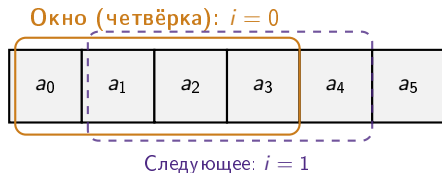
Аналогично парам, в заданиях могут встретиться **тройки** элементов (окно размера 3). Сдвиг происходит также по одному элементу.



```
# Для троек (окно из 3 элементов:  $3 - 1 = 2$ )  
for i in range(len(a) - 2):  
    x, y, z = a[i], a[i+1], a[i+2]
```

12: Четвёрки элементов (Окно = 4)

Четвёрки элементов образуют окно размера 4.



Для четвёрок удобнее объединять ячейки сразу в небольшие **списки (срезы)**, а не в отдельные переменные x , y , z , w . Так их проще обрабатывать и сортировать алгоритмами.

```
# Для четвёрок (окно из 4 элементов:  $4 - 1 = 3$ )  
for i in range(len(a) - 3):  
    t = [a[i], a[i+1], a[i+2], a[i+3]]
```

13: Сортировка срезов и упорядочивание

В задачах на четвёрки или тройки чисел могут возникать требования "числа идут по убыванию" или "разность между наибольшим и наименьшим".

Если вы сохранили срез в список `t`, задача решается в одну строчку:

```
for i in range(len(a) - 3):
    t = [a[i], a[i+1], a[i+2], a[i+3]]

    # Идут строго по убыванию:
    if t[0] > t[1] > t[2] > t[3]:
        ...

    # Разность максимального и минимального в 4-ке:
    diff = max(t) - min(t)
```

Задача 1 (№1993)

В файле содержится последовательность целых чисел от -10 000 до 10 000. Определите количество **пар** последовательности, в которых сумма элементов кратна 3 и не кратна 6, а произведение элементов оканчивается на 8. В ответе запишите **количество найденных пар**, затем **максимальную сумму элементов** таких пар.

Решение: Задача 1

```
with open('17.txt') as f:
    a = [int(x) for x in f]

ans = []
for i in range(len(a)-1):
    x, y = a[i], a[i+1]
    sm = x + y
    pr = x * y

    if sm % 3 == 0 and sm % 6 != 0 and abs(pr) % 10 == 8:
        ans.append(x + y)

print(len(ans), max(ans))
```

Задача 2 (№2400)

В файле содержится последовательность целых чисел от -10 000 до 10 000. Определите количество **пар** последовательности, в которых сумма двух элементов не менее 100, и хотя бы одно число отрицательное.

В ответе запишите количество найденных пар, затем максимальное из произведений элементов таких пар.

Решение: Задача 2

```
with open('17.txt') as f:
    a = [int(x) for x in f]

ans = []
for i in range(len(a)-1):
    x, y = a[i], a[i+1]

    if (x + y >= 100) and (x < 0 or y < 0):
        ans.append(x * y)

print(len(ans), max(ans))
```

Задача 3 (ФИПИ 133D72)

В файле содержится последовательность натуральных чисел (от 1 до 100 000). Определите количество **пар** последовательности, в которых остаток от деления на 55 хотя бы одного из элементов равен минимальному элементу последовательности.

В ответе запишите количество найденных пар, затем минимальную из сумм элементов таких пар.

Решение: Задача 3

```
with open('17.txt') as f:
    a = [int(x) for x in f]

# 1. Находим глобальный минимум
min_val = min(a)

ans = []
for i in range(len(a)-1):
    x, y = a[i], a[i+1]
    # 2. Проверяем условие
    if x % 55 == min_val or y % 55 == min_val:
        ans.append(x + y)

# 3. Выводим ответ
print(len(ans), min(ans))
```

Задача 4 (ФИПИ 45744А)

В файле содержится последовательность натуральных чисел от 1 до 100 000. Определите количество **пар** последовательности, в которых сумма остатков от деления обоих элементов на 15 равна минимальному элементу последовательности.

В ответе запишите количество пар, затем минимальную из сумм.

Решение: Задача 4

```
with open('17.txt') as f:
    a = [int(x) for x in f]

min_val = min(a)
ans = []

for i in range(len(a)-1):
    x, y = a[i], a[i+1]

    # Сумма остатков от деления на 15 равна минимальному элементу
    if (x % 15 + y % 15) == min_val:
        ans.append(x + y)

print(len(ans), min(ans))
```

Задача 5 (№1998)

В файле содержится последовательность целых чисел от -10 000 до 10 000. Определите количество **троек** элементов последовательности, в которых произведение элементов кратно 7, а сумма оканчивается на 5.

В ответе запишите **количество** троек, затем **максимальную** из сумм.

Решение: Задача 4

```
with open('17.txt') as f:
    a = [int(x) for x in f]

ans = []
for i in range(len(a)-2):
    x, y, z = a[i], a[i+1], a[i+2]
    pr = x * y * z
    sm = x + y + z

    # Модуль при взятии последней цифры обязателен для отрицательных чисел!
    if pr % 7 == 0 and abs(sm) % 10 == 5:
        ans.append(sm)

print(len(ans), max(ans))
```

Задача 6 (№2402)

Определите количество **троек** элементов последовательности (числа от 0 до 200), в которых **хотя бы одно** число в троичной системе счисления оканчивается на 2.

В ответе запишите количество троек, затем сумму минимальных чисел из таких троек.

Решение: Задача 6

```
with open('17.txt') as f:
    a = [int(x) for x in f]

ans = []
for i in range(len(a)-2):
    x, y, z = a[i], a[i+1], a[i+2]

    if x % 3 == 2 or y % 3 == 2 or z % 3 == 2:
        # Добавляем в массив минимальное из трех чисел текущей тройки
        ans.append(min(x, y, z))

print(len(ans), sum(ans))
```

Задача 7 (№2002)

В файле (числа от -10 000 до 10 000) определите количество **четвёрок** элементов последовательности, в которых числа идут строго **по убыванию**, а разность наибольшего и наименьшего строго больше 1000.

В ответе запишите **количество**, затем **минимальную сумму элементов**.

Решение: Задача 7

```
with open('17.txt') as f:
    a = [int(x) for x in f]

ans = []
for i in range(len(a)-3):
    t = [a[i], a[i+1], a[i+2], a[i+3]]

    # 1. Проверяем строгое убывание
    if t[0] > t[1] > t[2] > t[3]:
        # В отсортированном по убыванию списке t[0]=max, t[3]=min
        if max(t) - min(t) > 1000:
            ans.append(sum(t))

print(len(ans), min(ans))
```

Домашнее задание

Задача 1 (КЕГЭ №1999)

В файле содержится последовательность целых чисел от -10 000 до 10 000. Определите количество **троек** элементов последовательности, в которых хотя бы одно число кратно 12, а каждое число делится на 3.

В ответе запишите **количество найденных троек**, затем **минимальное из средних арифметических** таких троек (гарантируется, что среднее арифметическое в таких тройках целое число).

Ответы к домашнему заданию

Решение ДЗ 1 (№1999)

```
with open('17.txt') as f:
    a = [int(x) for x in f]

ans = []
for i in range(len(a)-2):
    x, y, z = a[i], a[i+1], a[i+2]
    cond1 = (x % 12 == 0) or (y % 12 == 0) or (z % 12 == 0)
    cond2 = (x % 3 == 0) and (y % 3 == 0) and (z % 3 == 0)

    if cond1 and cond2:
        ans.append((x + y + z) // 3)

print(len(ans), min(ans))
```

Домашнее задание

Задача 2 (ФИПИ 6ЕА065)

В файле содержится последовательность натуральных чисел от 1 до 100 000. Определите количество **пар** последовательности, в которых остаток от деления хотя бы одного из элементов на 16 равен минимальному элементу последовательности.

В ответе: количество пар, затем максимальная сумма таких пар.

Ответы к домашнему заданию

Решение ДЗ 2 (6EA065)

```
with open('17.txt') as f:
    a = [int(x) for x in f]

min_val = min(a)
ans = []

for i in range(len(a)-1):
    x, y = a[i], a[i+1]

    if x % 16 == min_val or y % 16 == min_val:
        ans.append(x + y)

print(len(ans), max(ans))
```

Домашнее задание

Задача 3 (ФИПИ 7478FE)

В файле содержится последовательность натуральных чисел от 1 до 100 000. Определите количество **пар** последовательности, в которых сумма остатков от деления обоих элементов на 21 равна минимальному элементу последовательности.

В ответе запишите количество найденных пар, затем максимальную сумму таких пар.

Ответы к домашнему заданию

Решение ДЗ 3 (7478FE)

```
with open('17.txt') as f:
    a = [int(x) for x in f]

min_val = min(a)
ans = []
for i in range(len(a)-1):
    x, y = a[i], a[i+1]
    if (x % 21 + y % 21) == min_val:
        ans.append(x + y)

print(len(ans), max(ans))
```

Домашнее задание

Задача 4 (ФИПИ 9С8813)

В файле содержится последовательность натуральных чисел от 1 до 100 000. Определите количество **пар** последовательности, в которых сумма остатков от деления обоих элементов на 18 равна минимальному элементу последовательности.

В ответе запишите **количество пар**, затем **максимальную сумму** таких пар.

Ответы к домашнему заданию

Решение ДЗ 4 (9C8813)

```
with open('17.txt') as f:
    a = [int(x) for x in f]

min_val = min(a)
ans = []
for i in range(len(a)-1):
    x, y = a[i], a[i+1]
    if (x % 18 + y % 18) == min_val:
        ans.append(x + y)

print(len(ans), max(ans))
```

Домашнее задание

Задача 5 (КЕГЭ №2401)

В файле содержится последовательность целых чисел от -10 000 до 10 000. Определите количество **пар** последовательности, в которых сумма модулей элементов больше или равна 50 и меньше или равна 200. Одно число может входить в несколько пар.

В ответе: количество найденных пар, причем минимальное из чисел, входящих в такие пары.

Ответы к домашнему заданию

Решение ДЗ 5 (№2401)

```
with open('17.txt') as f:
    a = [int(x) for x in f]

ans = []
for i in range(len(a)-1):
    x, y = a[i], a[i+1]
    s = abs(x) + abs(y)
    if 50 <= s <= 200:
        ans.append(min(x, y))

print(len(ans), min(ans))
```

Задание 17. Обработка числовой последовательности

Часть 2. Поиск глобальных параметров, тройки и четвёрки чисел

Информатика ЕГЭ